

Wait-free augmented static trie, recursive form

keys are $1 \dots N$, with N a power of two
every recursive component reached with bounds i and j covers exactly the keys $i \dots j$
the fields a value was built with are read back with a dot, as in $x.sum$
an internal component has the form $C[X, Y]$: one node with two recursive children

```

type versiontree:                                ▷ one of two shapes, immutable once built
1   vleaf(nat  $i$ , nat  $sum$ )                        ▷  $sum$  is 1 if  $i$  is in the set, else 0
2   vnode(nat  $i$ , nat  $j$ , nat  $sum$ , versiontree  $left$ , versiontree  $right$ )

type nodetree:                                    ▷ the skeleton, only its versions change
1   nleaf(nat  $i$ , versiontree  $version$ )            ▷  $version$  is the one mutable field
2   nnode(nat  $i$ , nat  $j$ , versiontree  $version$ , nodetree  $left$ , nodetree  $right$ )

versiontree snapshot(nodetree  $x$ ):                ▷ atomic read of the  $x$ 's version
1   return  $x.version$ 

nodetree build(nat  $i$ , nat  $j$ ):                    ▷ built once, with every  $sum$  at 0
1   if  $i = j$ 
2       return nleaf( $i$ , vleaf( $i$ , 0))
3    $l \leftarrow \text{build}(i, (i + j)/2)$ 
4    $r \leftarrow \text{build}((i + j)/2 + 1, j)$ 
5    $v_l \leftarrow \text{snapshot}(l)$ 
6    $v_r \leftarrow \text{snapshot}(r)$ 
7   return nnode( $i, j, 0, v_l, v_r, l, r$ )

Initialization:
1   nodetree  $root \leftarrow \text{build}(1, N)$           ▷ before any thread runs

bool refresh(nodetree  $x$ , nat  $i$ , nat  $j$ ):          ▷ one node reads its two child components
1    $old \leftarrow \text{read } x.version$ 
2    $vl \leftarrow \text{snapshot}(x.left)$ 
3    $vr \leftarrow \text{snapshot}(x.right)$ 
4   return CAS( $x.version, old, \text{vnode}(i, j, vl.sum + vr.sum, vl, vr)$ )

bool insert(nodetree  $x$ , nat  $i$ , nat  $j$ , key  $k$ ):    ▷ add  $k$ , true iff  $k$  was absent
1   if  $i = j$ 
2        $old \leftarrow \text{read } x.version$ 
3       if  $old.sum = 1$ 
4           return false
5       return CAS( $x.version, old, \text{vleaf}(i, 1)$ )
6   if  $k \leq (i + j)/2$ 
7        $result \leftarrow \text{insert}(x.left, i, (i + j)/2, k)$ 
8   else
9        $result \leftarrow \text{insert}(x.right, (i + j)/2 + 1, j, k)$ 
10  if not refresh( $x, i, j$ )
11      refresh( $x, i, j$ )                            ▷ two attempts always suffice
12  return  $result$ 

```

<pre> bool delete(nodetree x, nat i, nat j, key k): 1 if i = j 2 old ← read x.version 3 if old.sum = 0 4 return false 5 return CAS(x.version, old, vleaf(i, 0)) 6 if k ≤ (i + j)/2 7 result ← delete(x.left, i, (i + j)/2, k) 8 else 9 result ← delete(x.right, (i + j)/2 + 1, j, k) 10 if not refresh(x, i, j) 11 refresh(x, i, j) 12 return result </pre>	▷ as insert, but clears the leaf
<pre> bool find(key k): 1 return find_at(snapshot(root), 1, N, k) </pre>	▷ is k in the set
<pre> bool find_at(versiontree v, nat i, nat j, key k): 1 if i = j 2 return v.sum = 1 3 if k ≤ (i + j)/2 4 return find_at(v.left, i, (i + j)/2, k) 5 return find_at(v.right, (i + j)/2 + 1, j, k) </pre>	▷ plain sequential code, v is frozen
<pre> nat size(): 1 return snapshot(root).sum </pre>	▷ answered by a single read
<pre> int select(nat r): 1 v ← snapshot(root) 2 if v.sum < r 3 return -1 4 return select_at(v, 1, N, r) </pre>	▷ the rth smallest key, -1 if none
<pre> key select_at(versiontree v, nat i, nat j, nat r): 1 if i = j 2 return i 3 if r ≤ v.left.sum 4 return select_at(v.left, i, (i + j)/2, r) 5 return select_at(v.right, (i + j)/2 + 1, j, r - v.left.sum) </pre>	▷ assumes $r \leq v.sum$, so it finds one
<pre> nat rank(key k): 1 return rank_at(snapshot(root), 1, N, k) </pre>	▷ how many keys are at most k
<pre> nat rank_at(versiontree v, nat i, nat j, key k): 1 if i = j 2 return v.sum 3 if k ≤ (i + j)/2 4 return rank_at(v.left, i, (i + j)/2, k) 5 return v.left.sum + rank_at(v.right, (i + j)/2 + 1, j, k) </pre>	▷ the same count, taken inside v